

TEDx



UNIVERSITÀ
DEGLI STUDI
DI BERGAMO



Panoramica



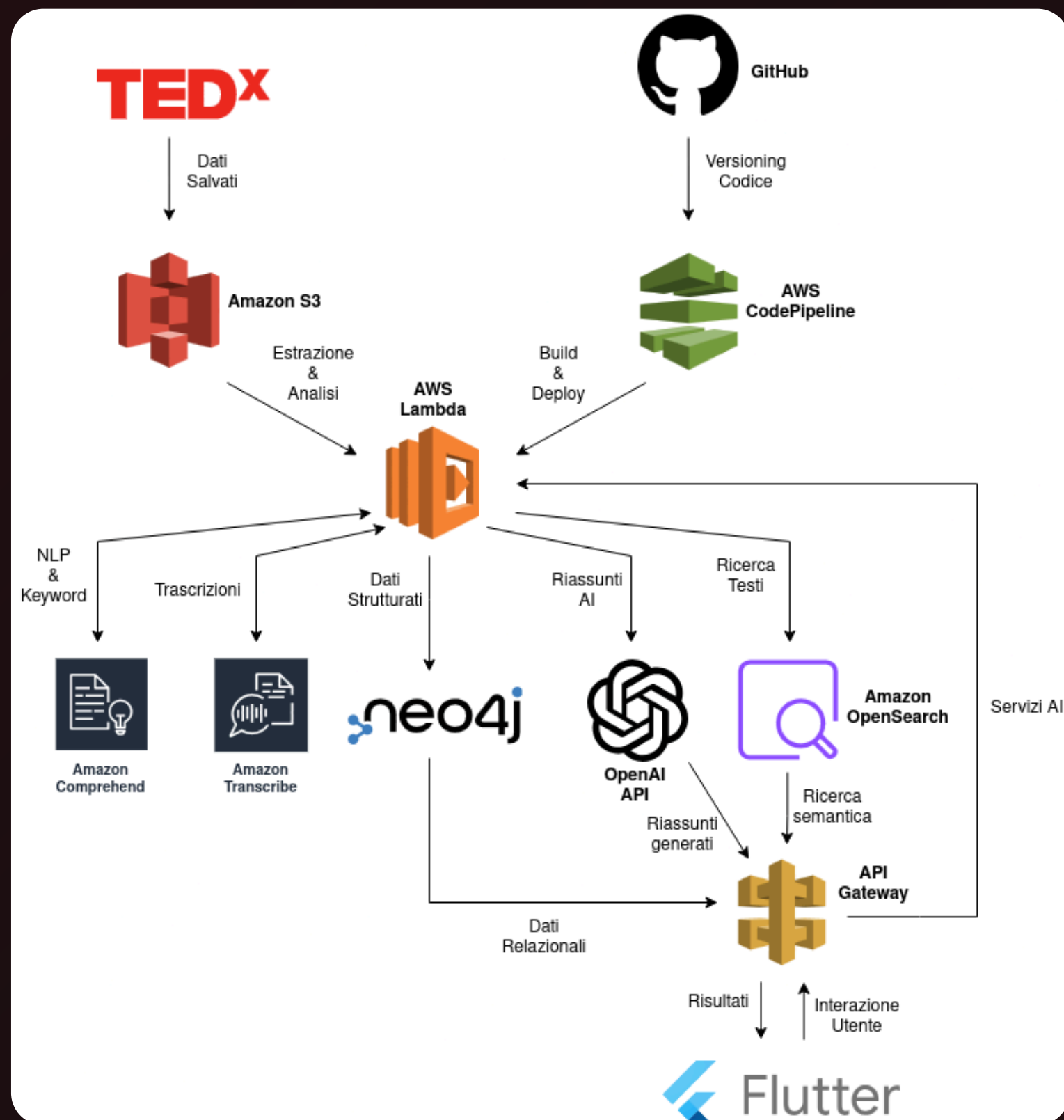
Scoperta Facilitata dei Talk

- *Estensione dell'applicazione "MyTEDx" con una feature che permette di vedere la lista dei video consigliati "Watch_Next" usando l'API sviluppata nella esercitazione n°3.*

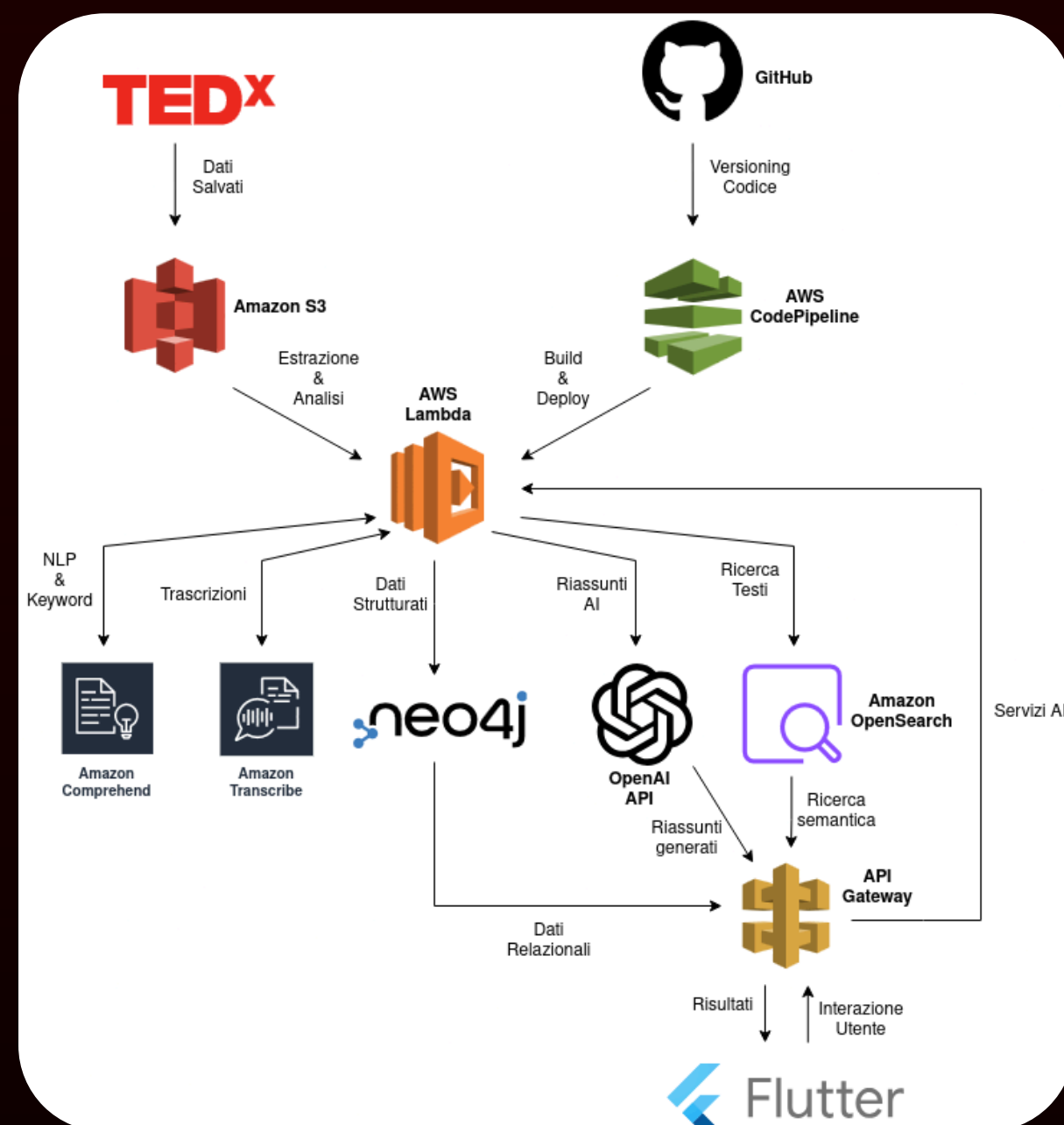


Nuova Funzionalità sulla Board

- *Implementazione dell'applicazione 'MyTEDx' tramite Flutter, con lo sviluppo dell'interfaccia utente (GUI) e la gestione delle chiamate alle API del backend.*



MEMO

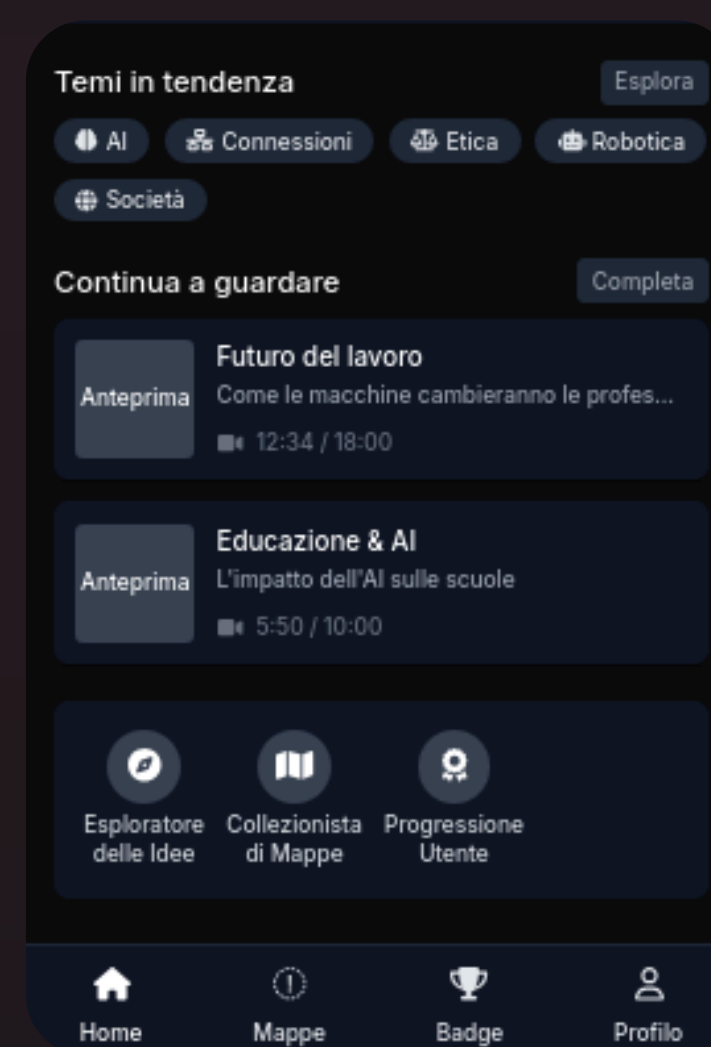
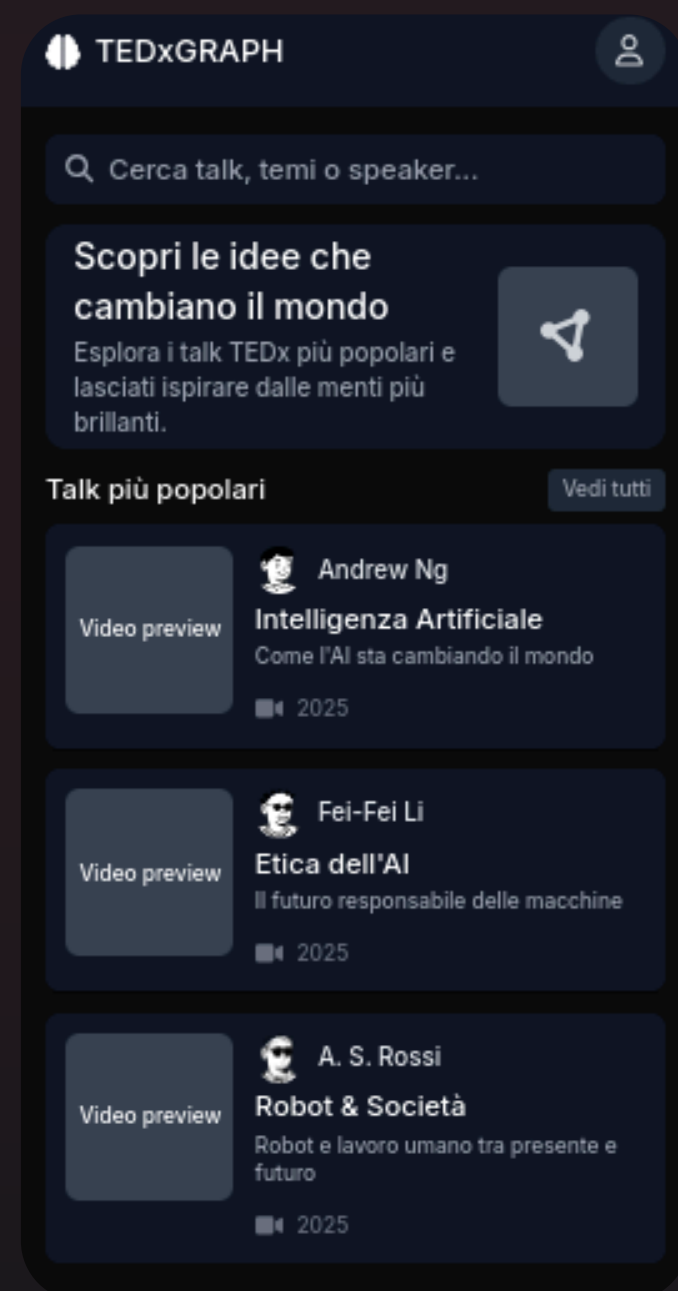


Implementazione full-stack con

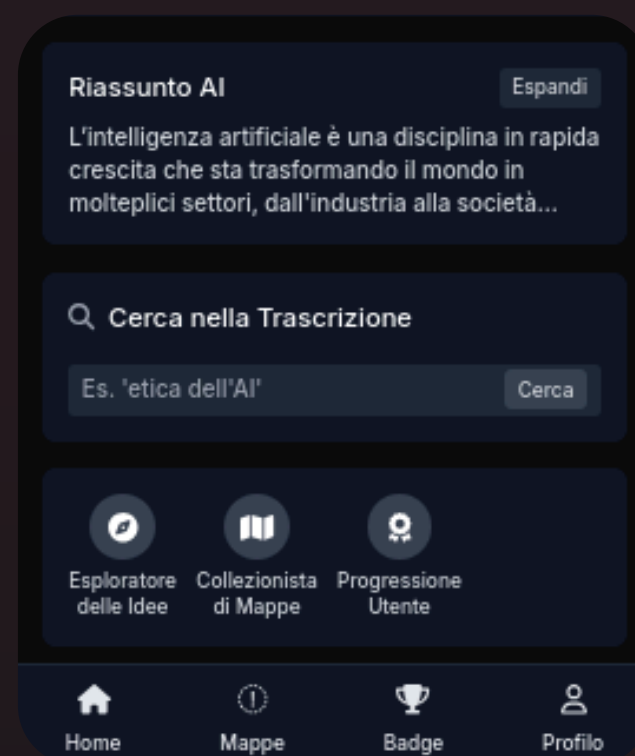
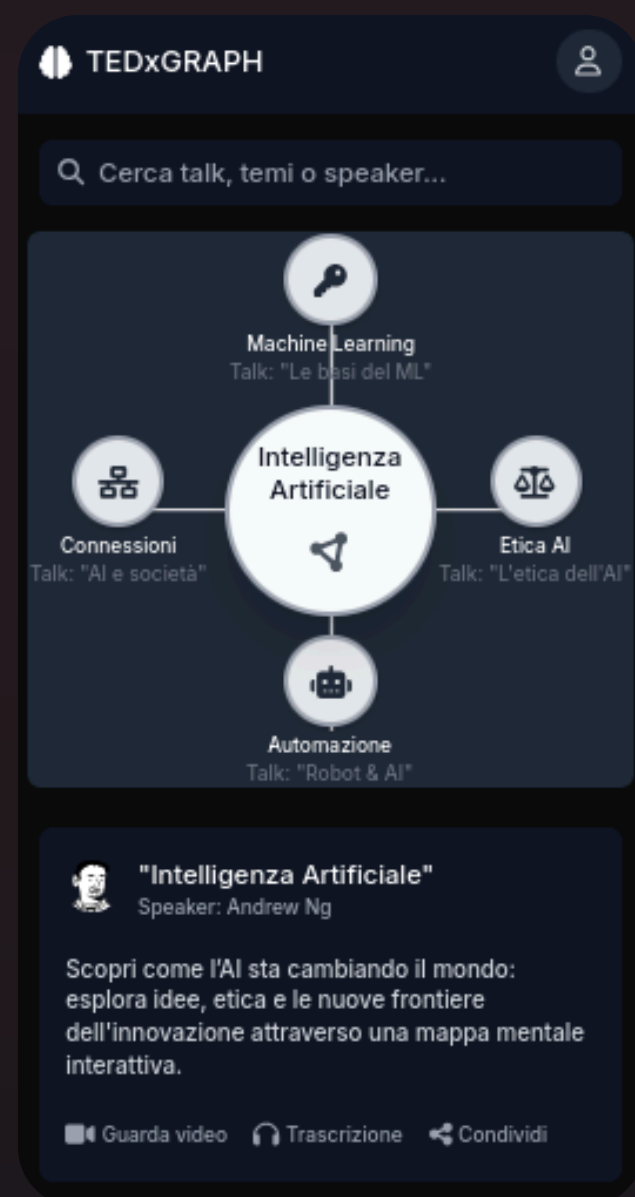
Sviluppo dell'interfaccia utente utilizzando Flutter, in collegamento con l'architettura già realizzata nelle prime tre esercitazioni. Dopo aver gestito l'estrazione, l'elaborazione e l'organizzazione dei dati tramite servizi cloud e AI, questa fase si concentra sull'integrazione del frontend Flutter per permettere l'interazione dell'utente con il sistema.



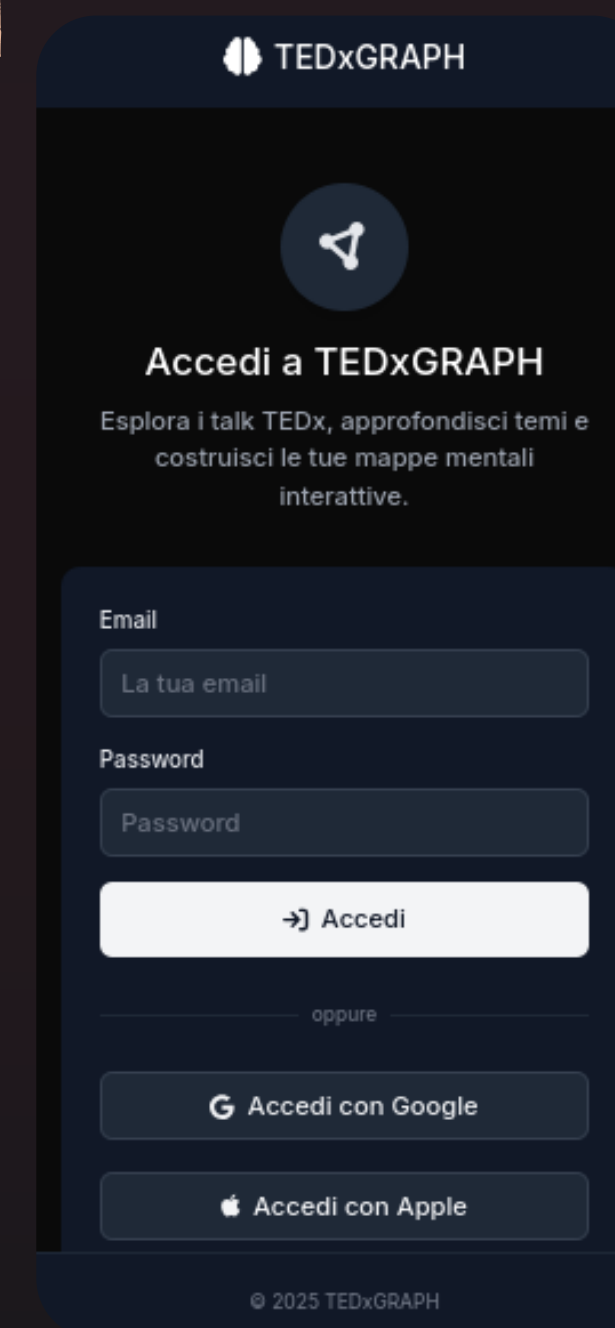
Homepage



Task



Login



```
tedx_graph/
├── core/                                # Configurazioni condivise e strumenti generici
│   ├── constants/
│   │   └── api_constants.dart          # Costanti per endpoint e parametri API
│   └── utils/
│       └── debouncer.dart              # Debouncer per ottimizzare le ricerche
├── data/                                # Gestione dei dati remoti e locali
│   ├── models/
│   │   ├── related_talk.dart           # Modello per talk correlati
│   │   ├── searched_talk.dart          # Modello per risultati ricerca
│   │   ├── talk_summary.dart           # Modello per riassunti dei talk
│   │   └── talk.dart                   # Modello generico del talk TEDx
│   └── services/
│       └── talk_api_service.dart        # Chiamate API per recupero dati da backend
├── navigation/
│   └── app_router.dart                  # Definizione delle rotte e navigazione
├── presentation/                         # Layer di presentazione UI + logica app
│   ├── providers/
│   │   ├── search_providers.dart       # Provider per gestire la ricerca dei talk
│   │   └── talk_providers.dart         # Provider per gestire lo stato dei talk
│   └── screens/
│       ├── global_search_screen.dart    # Schermata di ricerca globale
│       ├── home_screen.dart             # Schermata principale
│       ├── talk_detail_screen.dart      # Dettaglio di un talk selezionato
│       └── talks_by_tag_screen.dart     # Talk filtrati per tag/parola chiave
│   └── widgets/
│       ├── common_widgets.dart          # Componenti riutilizzabili (button, loader, ecc.)
│       ├── related_talk_tile.dart       # Widget per visualizzare un talk correlato
│       └── talk_card.dart               # Card per mostrare un talk nella lista
├── app_theme.dart                       # Tema globale (colori, tipografia, dark/light mode)
└── main.dart                            # Entry point principale dell'app Flutter
```

Albero del progetto (2/3)

<i>Cartella</i>	<i>Descrizione</i>
core/constants	<i>Contiene costanti globali (es. URL API, chiavi) usate in tutta l'applicazione.</i>
core/utils	<i>Funzioni di utilità generiche, come Debouncer per limitare le chiamate ripetute (es. nella ricerca).</i>
data/models	<i>Modelli dati per mappare oggetti JSON (talk, riassunti, risultati di ricerca, ecc.).</i>
data/services	<i>Servizi API per gestire le chiamate HTTP (es. recupero dati da API Gateway + Lambda).</i>
navigation	<i>Logica di routing centralizzata dell'app (gestisce le transizioni tra le schermate).</i>

Albero del progetto (3/3)

<i>Cartella</i>	<i>Descrizione</i>
presentation/providers	<i>Provider per la gestione dello stato usando Riverpod (es. provider per ricerca e talk).</i>
presentation/screens	<i>Tutte le schermate principali dell'app (home, search, detail, ecc.).</i>
presentation/widgets	<i>Componenti riutilizzabili come TalkCard, RelatedTalkTile, o widget comuni.</i>
app_theme.dart	<i>Tema personalizzato dell'applicazione (colori, font, stile globale).</i>
main.dart	<i>Entry point dell'app. Inizializza provider, routing, tema e avvia l'interfaccia.</i>


```
Windsurf: Refactor | Explain | Generate Docstring | x
def get_neo4j_driver():
    global driver
    if driver is None:
        if not NEO4J_URI or not NEO4J_USER or not NEO4J_PASSWORD:
            print("Errore: Variabili d'ambiente NEO4J_URI, NEO4J_USER, NEO4J_PASSWORD non configurate.")
            raise ValueError("Variabili d'ambiente Neo4j non configurate correttamente.")
        try:
            print(f"Tentativo di connessione a Neo4j URI: {NEO4J_URI}")
            driver = GraphDatabase.driver(NEO4J_URI, auth=basic_auth(NEO4J_USER, NEO4J_PASSWORD))
            # Verifica la connettività per assicurarsi che il driver sia funzionante
            driver.verify_connectivity()
            print("Connessione a Neo4j stabilita con successo.")
        except Exception as e:
            print(f"Errore durante la connessione a Neo4j: {e}")
            # Rilancia l'eccezione per essere gestita dal chiamante (lambda_handler)
            # Questo assicura che driver rimanga None se la connessione fallisce
            driver = None # Assicurati che rimanga None se fallisce
            raise ConnectionError(f"Impossibile connettersi a Neo4j: {e}") from e
    return driver
```

Crea e ritorna il driver Neo4j se non è già inizializzato, controllando le variabili d'ambiente e la connessione al database.

```
Windsurf: Refactor | Explain | x
def lambda_handler(event, context):
    """
    Funzione principale della Lambda.
    Si connette a Neo4j, recupera tutti i tag e li restituisce come JSON.
    """
    print(f"Evento ricevuto: {json.dumps(event)}")

    try:
        db_driver = get_neo4j_driver() # Ottiene o inizializza il driver

        # Utilizza una sessione per interagire con il database
        # La sessione viene chiusa automaticamente all'uscita dal blocco 'with'
        with db_driver.session() as session:
            # Esegue la transazione in modalità lettura
            tag_list = session.read_transaction(get_all_tags)

        print(f"Tag recuperati: {tag_list}")

        # Costruisce la risposta HTTP di successo
        # Il corpo della risposta è una stringa JSON contenente la lista dei tag
        return {
            'statusCode': 200,
            'headers': {
                'Content-Type': 'application/json', # Fondamentale per indicare al client il tipo di contenuto
                'Access-Control-Allow-Origin': '*' # Permette richieste CORS da qualsiasi origine (da restringere)
            },
            'body': json.dumps(tag_list) # Serializza la lista di tag in una stringa JSON
        }

    except ValueError as ve: # Errore di configurazione
        print(f"Errore di configurazione: {ve}")
        return {
            'statusCode': 500, # Internal Server Error (o 400 Bad Request se l'errore è dovuto a input)
            'headers': {'Content-Type': 'application/json'},
            'body': json.dumps({'error': 'Errore di configurazione del servizio', 'details': str(ve)})
        }
```

```
Windsurf: Refactor | Explain | x
def get_all_tags(tx):
    """
    Esegue una query Cypher per ottenere tutti i tag distinti dai nodi.
    """
    # Query per estrarre tutti i tag unici dai nodi che hanno una proprietà 'tags'
    # La proprietà 'tags' è assunta essere una lista di stringhe.
    query = """
    MATCH (n)
    WHERE n.tags IS NOT NULL AND size(n.tags) > 0 // Assicura che esista e non sia vuota
    UNWIND n.tags AS tag // Scompatta la lista di tags
    RETURN DISTINCT tag // Restituisce solo i tag unici
    ORDER BY tag // Ordina i tag alfabeticamente
    """
    result = tx.run(query)
    # Estrae il valore 'tag' da ogni record del risultato
    return [record["tag"] for record in result]
```

Esegue una query Neo4j per estrarre e restituire tutti i tag unici presenti nella proprietà tags dei nodi.


```
Windsurf: Refactor | Explain | Generate Docstring | ×
def get_talks_by_tags(tx, tags):
    query = """
    MATCH (t:Talk)
    WHERE t.tags IS NOT NULL AND ANY(tag IN $tags WHERE tag IN t.tags)
    RETURN id(t) AS id, t.title AS title, t.speakers AS speakers, t.description AS description, t.tags AS tags
    LIMIT 20
    """
    result = tx.run(query, tags=tags)
    return [
        {
            "id": record["id"],
            "title": record["title"],
            "speakers": record["speakers"],
            "description": record["description"],
            "tags": record["tags"]
        }
        for record in result
    ]
```

Crea il driver Neo4j leggendo le variabili d'ambiente. Verifica la connessione e solleva errore se fallisce.

```
def get_connected_nodes(tx, node_id_param):
    query = (
        "MATCH (startNode {id: $node_id_param})-->(connectedNode) "
        "RETURN connectedNode.title AS title, "
        "        connectedNode.speakers AS speakers, "
        "        connectedNode.description AS description"
    )
    result = tx.run(query, node_id_param=node_id_param)

    nodes_data = []
    for record in result:
        nodes_data.append({
            "title": record["title"],
            "speakers": record["speakers"], # Assumendo sia una lista o una stringa
            "description": record["description"]
        })
    return nodes_data
```

Trova i nodi collegati a quello con l'ID specificato. Ritorna titolo, speaker e descrizione.

```
Windsurf: Refactor | Explain | Generate Docstring | ×
def get_talks_by_tags(tx, tags):
    query = """
    MATCH (t:Talk)
    WHERE t.tags IS NOT NULL AND ANY(tag IN $tags WHERE tag IN t.tags)
    RETURN id(t) AS id, t.title AS title, t.speakers AS speakers, t.description AS description, t.tags AS tags
    LIMIT 20
    """
    result = tx.run(query, tags=tags)
    return [
        {
            "id": record["id"],
            "title": record["title"],
            "speakers": record["speakers"],
            "description": record["description"],
            "tags": record["tags"]
        }
        for record in result
    ]
```

Cerca nodi Talk che contengono almeno uno dei tag dati. Ritorna i primi 20 risultati con i dati principali.

```
Windsurf: Refactor | Explain | ×
def lambda_handler(event, context):
    """
    Funzione principale della Lambda.
    Si connette a Neo4j, recupera tutti i tag e li restituisce come JSON.
    """
    print(f"Evento ricevuto: {json.dumps(event)}")

    try:
        db_driver = get_neo4j_driver() # Ottiene o inizializza il driver

        # Utilizza una sessione per interagire con il database
        # La sessione viene chiusa automaticamente all'uscita dal blocco 'with'
        with db_driver.session() as session:
            # Esegue la transazione in modalità lettura
            tag_list = session.read_transaction(get_all_tags)

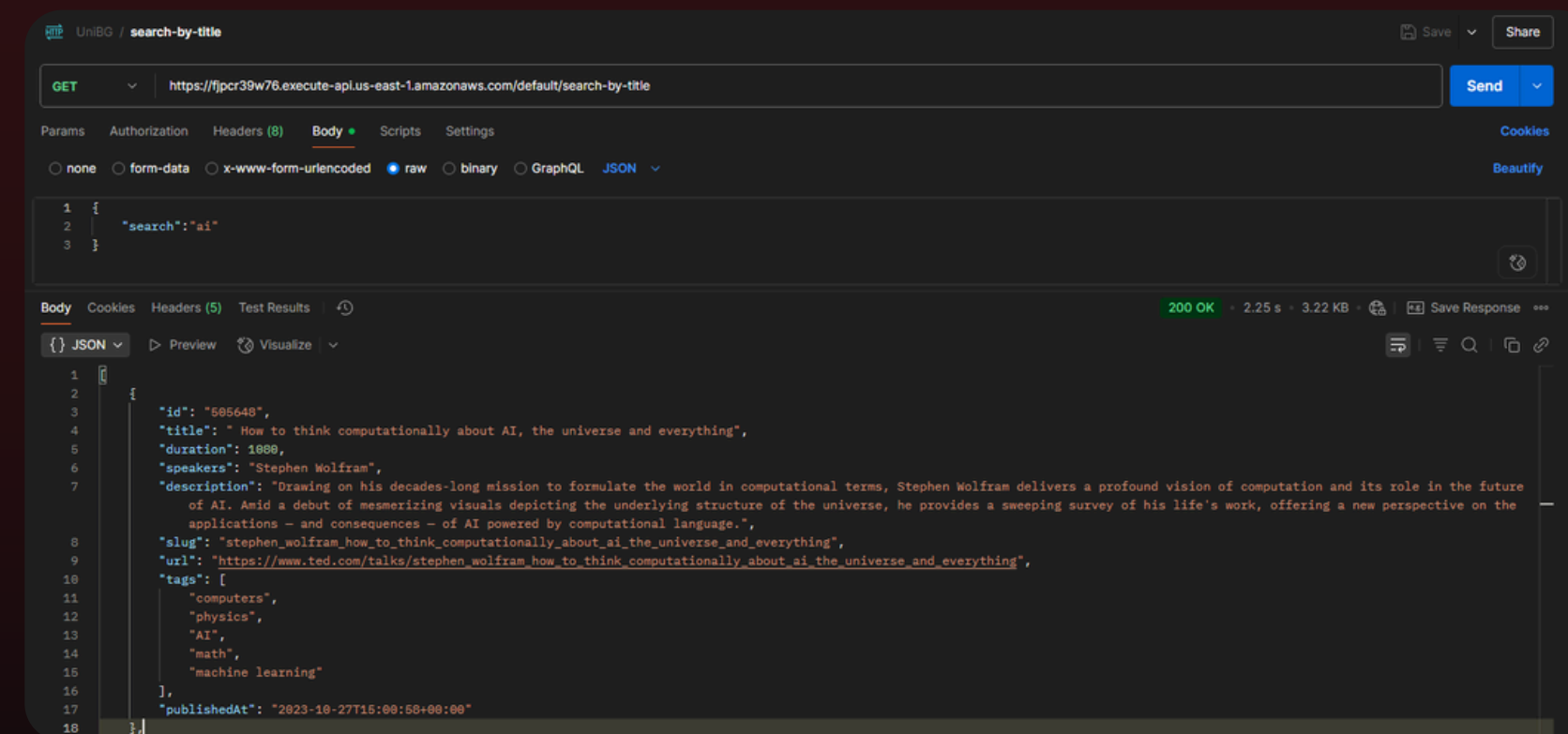
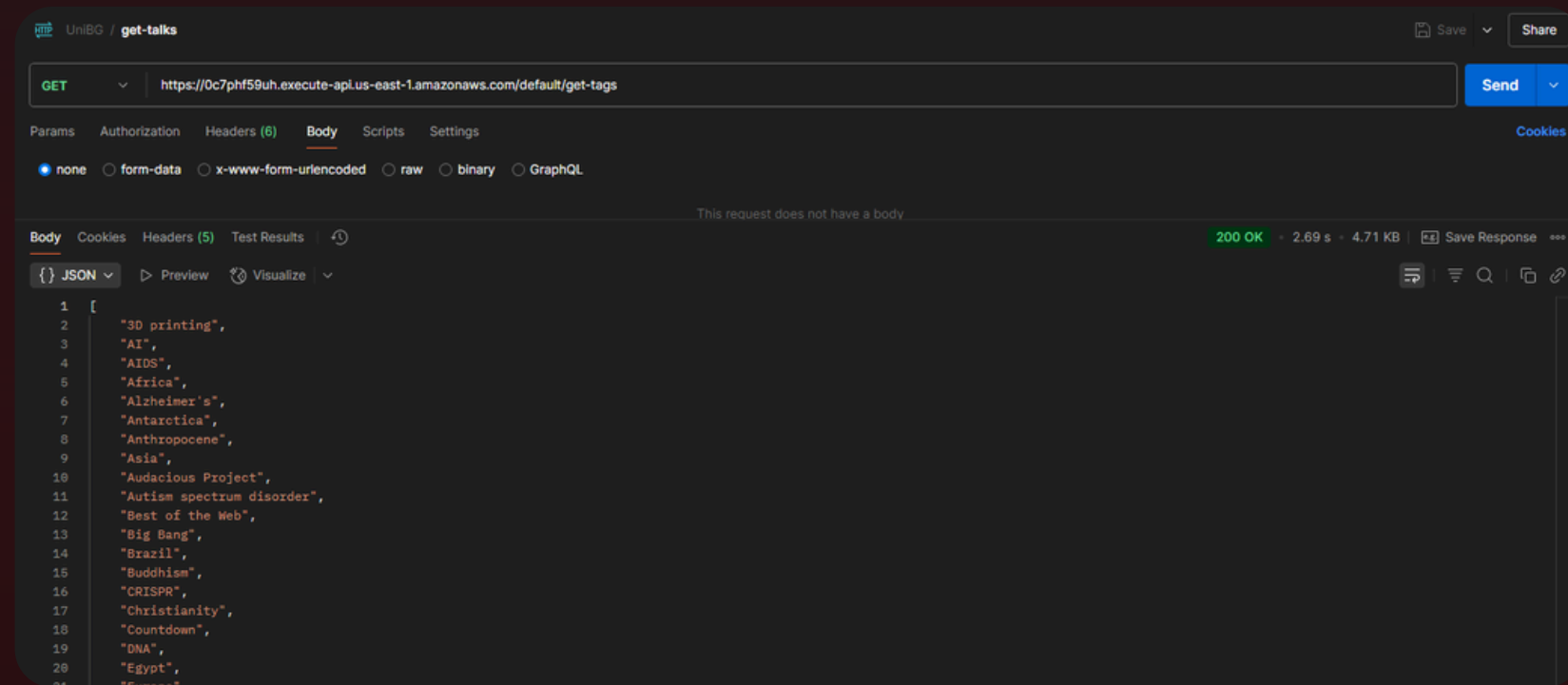
        print(f"Tag recuperati: {tag_list}")

        # Costruisce la risposta HTTP di successo
        # Il corpo della risposta è una stringa JSON contenente la lista dei tag
        return {
            'statusCode': 200,
            'headers': {
                'Content-Type': 'application/json', # Fondamentale per indicare al client il tipo di contenuto
                'Access-Control-Allow-Origin': '*' # Permette richieste CORS da qualsiasi origine (da restringere)
            },
            'body': json.dumps(tag_list) # Serializza la lista di tag in una stringa JSON
        }

    except ValueError as ve: # Errore di configurazione
        print(f"Errore di configurazione: {ve}")
        return {
            'statusCode': 500, # Internal Server Error (o 400 Bad Request se l'errore è dovuto a input)
            'headers': {'Content-Type': 'application/json'},
            'body': json.dumps({'error': 'Errore di configurazione del servizio', 'details': str(ve)})
        }
```

Legge id o tags dalla richiesta e interroga Neo4j. Ritorna i nodi trovati o un errore se mancano i parametri.

Testing



```
# Variabili d'ambiente (da configurare nella Lambda)
NEO4J_URI = os.environ.get('NEO4J_URI')
NEO4J_USER = os.environ.get('NEO4J_USER')
NEO4J_PASSWORD = os.environ.get('NEO4J_PASSWORD')

driver = None

Windsurf: Refactor | Explain | Generate Docstring | ×
def get_neo4j_driver():
    global driver
    if driver is None:
        try:
            driver = GraphDatabase.driver(NEO4J_URI, auth=basic_auth(NEO4J_USER, NEO4J_PASSWORD))
            # Verifica la connessione (opzionale ma utile per il debug iniziale)
            driver.verify_connectivity()
            print("Successfully connected to Neo4j.")
        except Exception as e:
            print(f"Error connecting to Neo4j: {e}")
            # Se la connessione fallisce, driver rimarrà None e gestito dopo
            # Potresti voler sollevare un'eccezione qui se la connessione è critica all'avvio
            raise ConnectionError(f"Failed to connect to Neo4j: {e}") from e
    return driver
```

```
# Ottieni il driver
db_driver = get_neo4j_driver()

connected_nodes_list = []
with db_driver.session() as session:
    connected_nodes_list = session.read_transaction(get_connected_nodes, node_id)

print(f"Found {len(connected_nodes_list)} connected nodes.")

return {
    'statusCode': 200,
    'headers': {
        'Content-Type': 'application/json',
        'Access-Control-Allow-Origin': '*' # Importante per le app mobili/web
    },
    'body': json.dumps(connected_nodes_list)
}

except ConnectionError as ce:
    print(f"Connection Error: {ce}")
    return {
        'statusCode': 503, # Service Unavailable
        'headers': {'Content-Type': 'application/json'},
        'body': json.dumps({'error': 'Database connection failed', 'details': str(ce)})
    }

except Exception as e:
    print(f"Error processing request: {e}")
    # Includere str(e) può esporre dettagli, valuta per produzione
    return {
        'statusCode': 500,
        'headers': {'Content-Type': 'application/json'},
        'body': json.dumps({'error': 'Internal server error', 'details': str(e)})
    }
```

```
Windsurf: Refactor | Explain | Generate Docstring | ×
def get_talks_by_tags(tx, tags):
    query = """
    MATCH (t:Talk)
    WHERE t.tags IS NOT NULL AND ANY(tag IN $tags WHERE tag IN t.tags)
    RETURN id(t) AS id, t.title AS title, t.speakers AS speakers, t.description AS description, t.tags AS tags
    LIMIT 20
    """
    result = tx.run(query, tags=tags)
    return [
        {
            "id": record["id"],
            "title": record["title"],
            "speakers": record["speakers"],
            "description": record["description"],
            "tags": record["tags"]
        }
        for record in result
    ]
```

```
Windsurf: Refactor | Explain | Generate Docstring | ×
def get_connected_nodes(tx, node_id_param):
    query = (
        "MATCH (startNode {id: $node_id_param})-->(connectedNode) "
        "RETURN connectedNode.id AS id, "
        "       connectedNode.url AS url, "
        "       connectedNode.title AS title, "
        "       connectedNode.speakers AS speakers, "
        "       connectedNode.description AS description"
    )
    result = tx.run(query, node_id_param=node_id_param)

    nodes_data = []
    for record in result:
        nodes_data.append({
            "id": record["id"],
            "title": record["title"],
            "url": record["url"],
            "speakers": record["speakers"], # Assumendo sia una lista o una stringa
            "description": record["description"]
        })
    return nodes_data
```

```
TextField(
  controller: _searchController,
  decoration: InputDecoration(
    hintText: 'Enter talk title...',
    prefixIcon: const Icon(Icons.search),
    // ...
  ),
),
// ...
searchResultsAsync.when(
  data: (results) {
    if (results.isEmpty) {
      return EmptyContent(message: 'No talks found for "${_searchController.text}");
    }
    return ListView.builder(
      itemCount: results.length,
      itemBuilder: (context, index) {
        final searchedTalk = results[index];
        return ListTile(
          title: Text(searchedTalk.title),
          onTap: () {
            final partialTalk = Talk(
              id: searchedTalk.id,
              title: searchedTalk.title,
              details: 'Loading details...',
              slug: searchedTalk.id,
              mainSpeaker: 'Loading speaker...',
              url: '',
            );
            context.push('${AppRouter.talkDetailRoute}/${searchedTalk.id}', extra: partialTalk);
          },
        );
      },
    );
  },
  loading: () => const AppLoader(),
  error: (error, stack) => ErrorDisplay(message: error.toString()),
)
```

```
Future<List<Talk>> getTalksByTag(String tag, int page) async {
  final url = Uri.parse(ApiConstants.getTalksByTagEndpoint);
  final response = await _client.post(
    url,
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'tag': tag, 'page': page, 'doc_per_page': ApiConstants.talksPerPage}),
  );
  if (response.statusCode == 200) {
    final List<dynamic> jsonList = json.decode(utf8.decode(response.bodyBytes));
    return jsonList.map((json) => Talk.fromJSON(json)).toList();
  } else {
    throw Exception('Failed to load talks by tag. Status: ${response.statusCode}');
  }
}
```

```
GoRoute(
  path: '$talkDetailRoute/:talkId',
  builder: (BuildContext context, GoRouterState state) {
    final talkId = state.pathParameters['talkId']!;
    final talk = state.extra as Talk?;
    if (talk == null) {
      return Scaffold(
        appBar: AppBar(title: const Text("Error")),
        body: const Center(child: Text("Talk data not provided to detail screen.")),
      );
    }
    return TalkDetailScreen(talk: talk);
  },
),
```



```
class TalkCard extends StatelessWidget {
  final Talk talk;
  // ...
  @override
  Widget build(BuildContext context) {
    // ...
    return Card(
      // ...
      child: InkWell(
        onTap: () {
          // Navigazione al dettaglio usando talk.id
          context.push('${AppRouter.talkDetailRoute}/${talk.id}', extra: talk);
        },
        // ...
        child: Padding(
          // ...
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: <Widget>[
              Text(talk.title, style: theme.textTheme.titleLarge?.copyWith(fontWeight: FontWeig
              // ...
              Text(talk.mainSpeaker, style: theme.textTheme.titleMedium?.copyWith(color: theme.
              // ...
              Text(talk.details, style: theme.textTheme.bodyMedium, maxLines: 3, overflow: Text
              if (talk.keyPhrases.isNotEmpty) ...[
                // Mostra le keyphrases come chip
                Wrap(
                  children: talk.keyPhrases.take(5).map((phrase) => Chip(label: Text(phrase))).
                ),
              ],
            ],
          ),
        ),
      ),
    );
  }
}
```

```
final talksByTagProvider = StateNotifierProvider.family<
  TalksByTagNotifier,
  AsyncValue<List<Talk>>,
  String
>((ref, tag) {
  return TalksByTagNotifier(ref.watch(talkApiServiceProvider), tag);
});

class TalksByTagNotifier extends StateNotifier<AsyncValue<List<Talk>>> {
  // ...
  Future<void> fetchInitialTalks() async {
    // ...
    final talks = await _apiService.getTalksByTag(_tag, _currentPage);
    // ...
    state = AsyncValue.data(talks);
  }
  // ...
}
```

```
except json.JSONDecodeError:
    return f"Servizio di riassunto temporaneamente non disponibile (503). Dettagli: {response.text[:200]}" # Mostra parte del testo se non è JSON
return "Servizio di riassunto temporaneamente non disponibile (503)." # Fallback

else:
    print(f"Errore HTTP {response.status_code} da Hugging Face API (riassunto). Dettagli: {response.text}")
    return f"Errore {response.status_code} dal servizio di riassunto. Dettagli: {response.text[:200]}"

except requests.exceptions.Timeout:
    print("Timeout durante la chiamata a Hugging Face API per riassunto.")
    return "Il servizio di riassunto ha impiegato troppo tempo a rispondere."

except requests.exceptions.RequestException as req_err:
    print(f"Errore di richiesta generico con Hugging Face API per riassunto: {req_err}")
    return "Errore di connessione con il servizio di riassunto."

except Exception as e:
    print(f"Errore generico non gestito durante la generazione del riassunto con Hugging Face: {e}")
    import traceback
    traceback.print_exc()
    return "Errore imprevisto durante la generazione del riassunto."
```

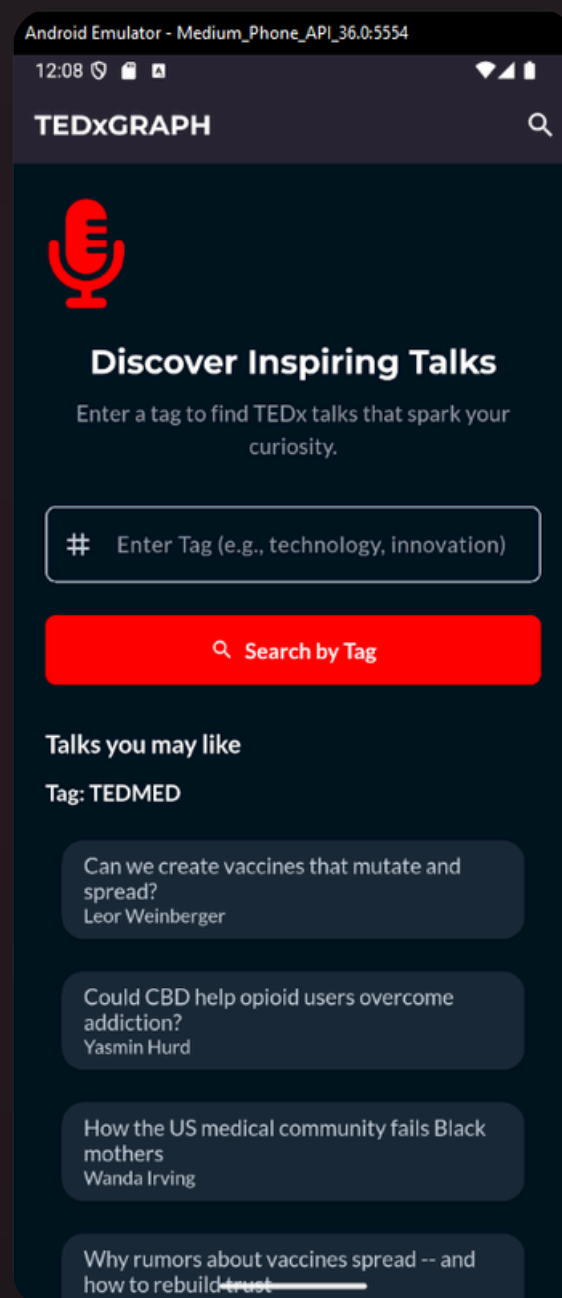
```
class TalkDetailScreen extends ConsumerWidget {
  final Talk talk;
  // ...
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    // ...
    final summaryAsync = ref.watch(talkSummaryProvider(talk.id));
    final relatedTalksAsync = ref.watch(relatedTalksProvider(talk.id));
    // ...
    return Scaffold(
      // ...
      body: SingleChildScrollView(
        // ...
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: <Widget>[
            Text(talk.title, style: theme.textTheme.displaySmall?.copyWith(color: theme.colorSc
            // ...
            // Riassunto AI
            summaryAsync.when(
              data: (summaryData) => Text(summaryData.summary, style: theme.textTheme.bodyMediu
              loading: () => const Center(child: AppLoader(size: 25)),
              error: (err, stack) => Text('Could not load summary: ${err.toString()}'),
            ),
            // ...
            // Talk correlati
            relatedTalksAsync.when(
              data: (relatedTalksList) {
                if (relatedTalksList.isEmpty) {
                  return const Text('No related talks found.');
```

```
class Talk {
  final String id; // ID unico (MongoDB _id o id)
  final String title;
  final String details; // Descrizione
  final String slug;
  final String mainSpeaker; // Speaker principale
  final String url;
  final List<String> keyPhrases;
  final String? publishedAt; // Data pubblicazione
  final int? duration; // Durata in secondi

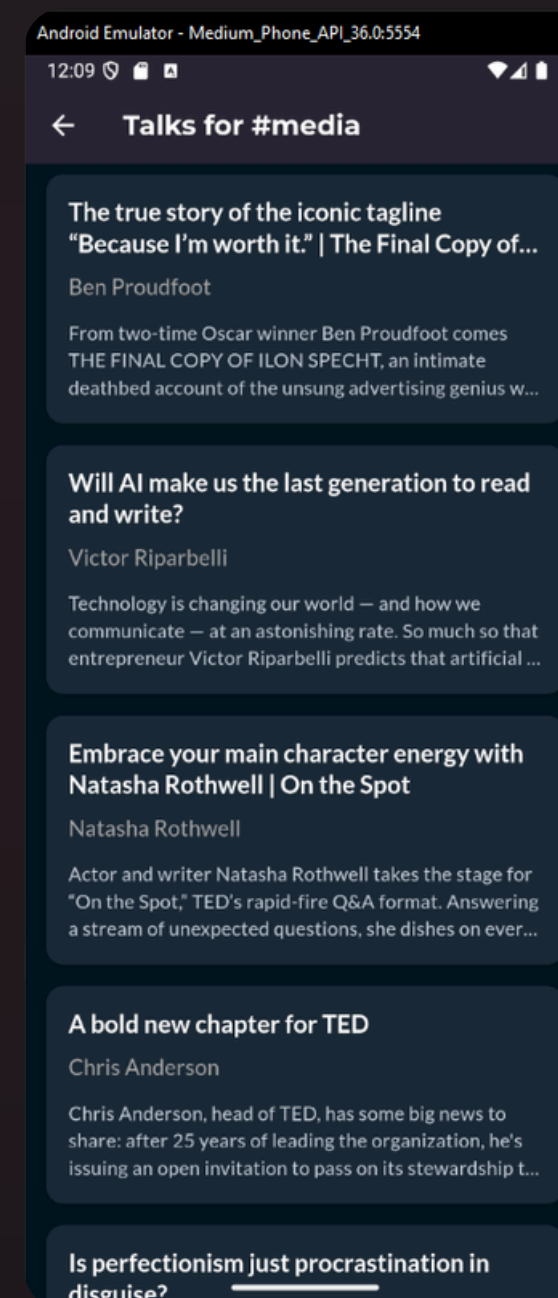
  // ...costruttore...

  factory Talk.fromJSON(Map<String, dynamic> jsonMap) {
    // Gestione robusta dell'ID e fallback
    String determineId(Map<String, dynamic> json) {
      if (json.containsKey('_id') && json['_id'] != null)
        return json['_id'].toString();
      if (json.containsKey('id') && json['id'] != null)
        return json['id'].toString();
      // Fallback su slug o timestamp
      return json['slug'] ?? DateTime.now().millisecondsSinceEpoch.toString();
    }

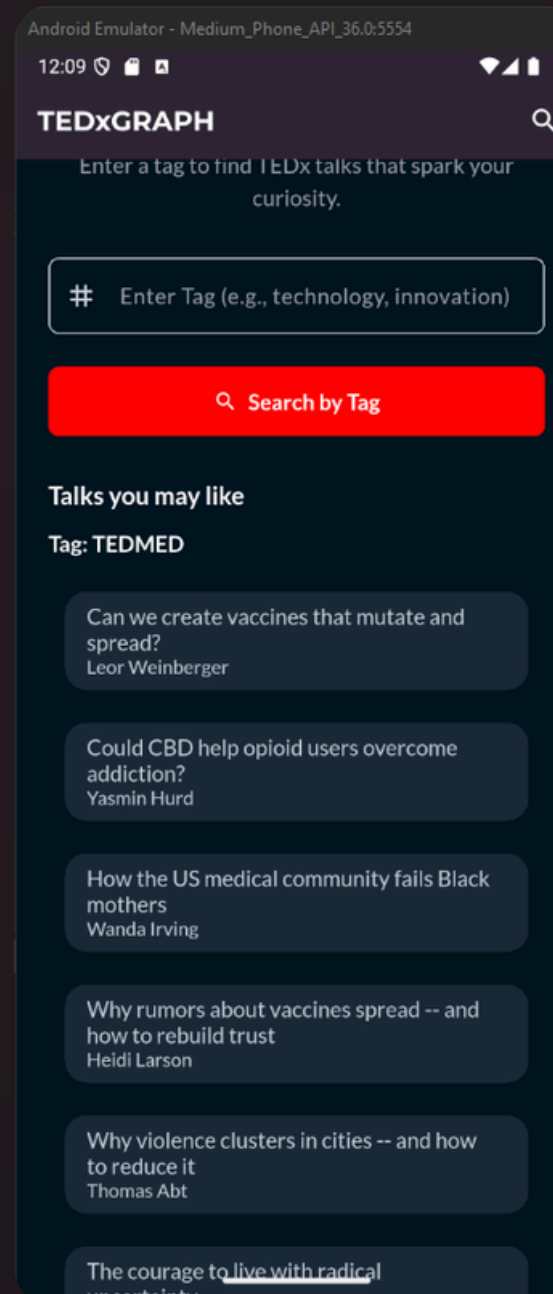
    return Talk(
      id: determineId(jsonMap),
      title: jsonMap['title'] ?? 'No Title',
      details: jsonMap['description'] ?? jsonMap['details'] ?? 'No Description',
      slug: (jsonMap['slug'] ?? ''),
      mainSpeaker: (jsonMap['speakers'] ?? jsonMap['mainSpeaker'] ?? "Unknown Speaker"),
      url: (jsonMap['url'] ?? ''),
      keyPhrases: (jsonMap['comprehend_analysis']?['KeyPhrases'] as List<dynamic>?)
        ?.map((e) => e.toString()).toList()
        ?? (jsonMap['keyPhrases'] as List<dynamic>?)?.map((e) => e.toString()).toList()
        ?? [],
      publishedAt: jsonMap['publishedAt']?.toString(),
      duration: jsonMap['duration'] is int
        ? jsonMap['duration']
        : (jsonMap['duration'] is String ? int.tryParse(jsonMap['duration']) : null),
    );
  }
}
```

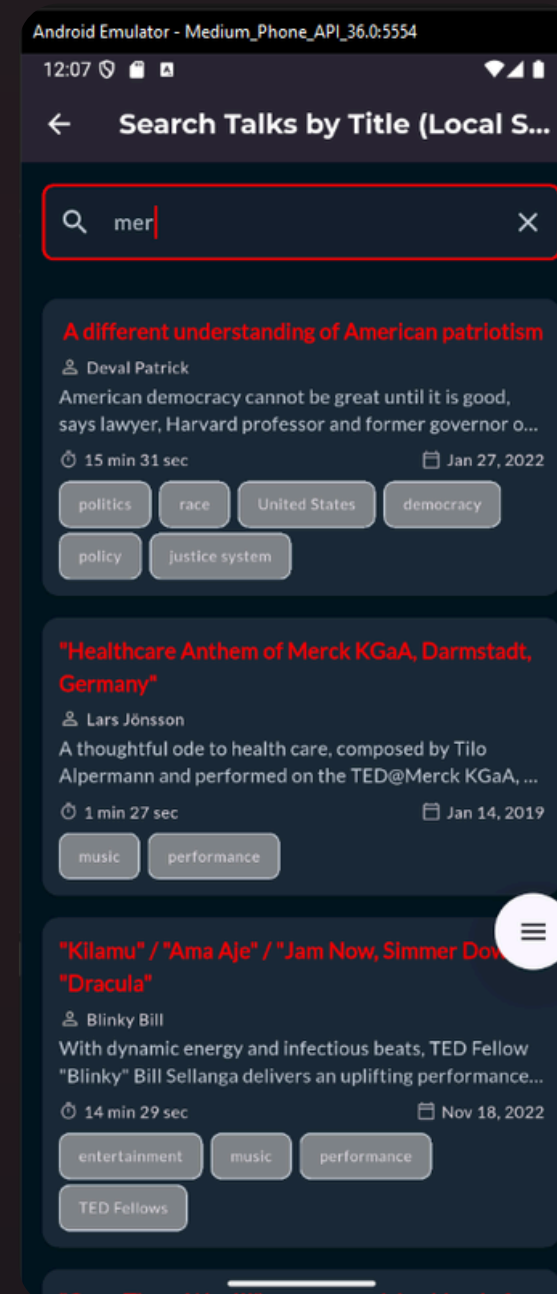
HomePage



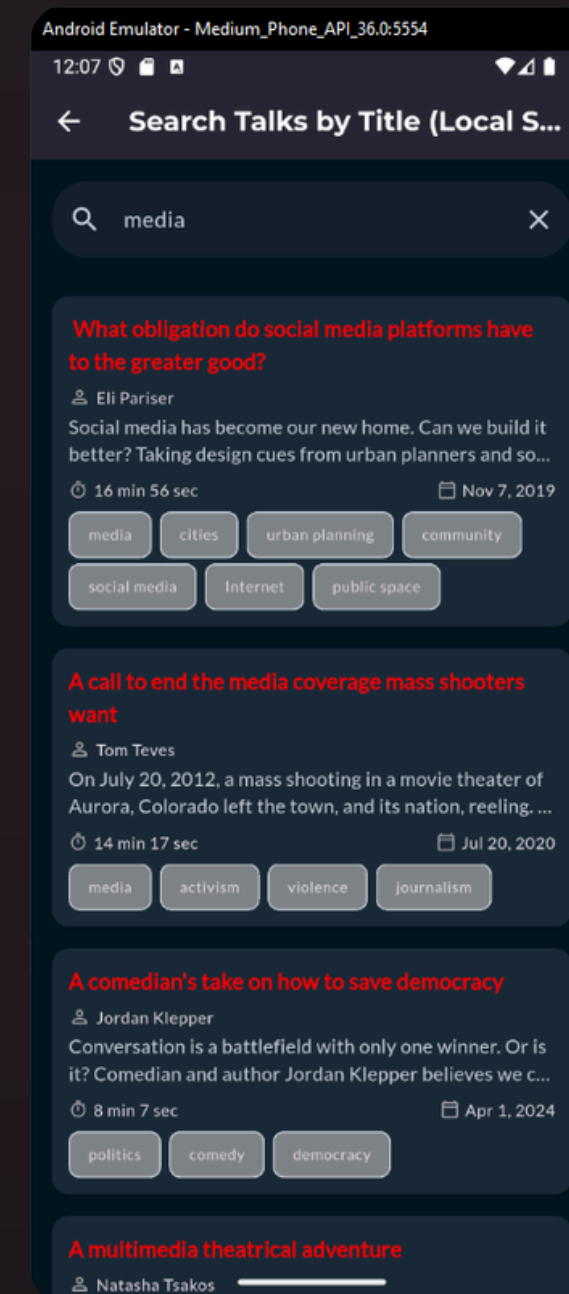
Search talks by tag and view suggested talks



Search talks by tag and view suggested talks



Automatic talk search by title



Talk search by Title

Android Emulator - Medium_Phone_API_36.0:5554

12:05

← A call to end the media cover...

A call to end the media coverage mass shooters want

🎤 Tom Teves

📅 20/07/2020 ⌚ 14:17

📌 About this Talk

On July 20, 2012, a mass shooting in a movie theater of Aurora, Colorado left the town, and its nation, reeling. To many -- including Tom Teves, who lost his son in the tragedy -- the news coverage that followed focused on all the wrong things. Why did the reporting overwhelmingly fixate on the shooter rather than the lives of the victims or the heroic efforts of first responders? With urgency and measure, Teves calls for responsible media attention that acts in the interest of the public (instead of profit) by revoking what shooters want most: infamy.

🔥 AI Summary

1. The speaker discusses the tragedy of losing his son, Alex, in the 2012 Aurora movie theater shooting.
2. He argues that the media's coverage of mass shootings, focusing on the shooter's name and image, unwittingly contributes to the perpetuation of such events by providing the shooters with the notoriety they seek.
3. The speaker introduces "No Notoriety," an initiative aimed at challenging the media to adhere to research-backed principles that minimize the shooter's name and

Talk Detail with openapi

Android Emulator - Medium_Phone_API_36.0:5554

12:06

← A call to end the media cover...

things. Why did the reporting overwhelmingly fixate on the shooter rather than the lives of the victims or the heroic efforts of first responders? With urgency and measure, Teves calls for responsible media attention that acts in the interest of the public (instead of profit) by revoking what shooters want most: infamy.

🔥 AI Summary

1. The speaker discusses the tragedy of losing his son, Alex, in the 2012 Aurora movie theater shooting.
2. He argues that the media's coverage of mass shootings, focusing on the shooter's name and image, unwittingly contributes to the perpetuation of such events by providing the shooters with the notoriety they seek.
3. The speaker introduces "No Notoriety," an initiative aimed at challenging the media to adhere to research-backed principles that minimize the shooter's name and image while promoting the names and images of victims, heroes, and first responders.
4. He emphasizes that this is not censorship but a request for responsible journalistic practices, similar to those already in place for journalists who are kidnapped or victims of sexual assault or suicide.
5. The speaker calls on the public to hold media accountable by reducing their viewership, clicks, and engagement with content that gratuitously uses the names and images of shooters, and to write to news organizations and advertisers expressing their concerns.
6. The speaker concludes by urging the public to take action to reduce random mass shootings, as they have the power to do so.

🔥 Related Talks

Why violence is rising with global temperatures
Peter Schwartzstein
Climate change doesn't just melt ice caps. It also fuels

Talk Detail with openapi

Android Emulator - Medium_Phone_API_36.0:5554

12:06

← A call to end the media cover...

already in place for journalists who are kidnapped or victims of sexual assault or suicide.

5. The speaker calls on the public to hold media accountable by reducing their viewership, clicks, and engagement with content that gratuitously uses the names and images of shooters, and to write to news organizations and advertisers expressing their concerns.

6. The speaker concludes by urging the public to take action to reduce random mass shootings, as they have the power to do so.

🔥 Related Talks

Why violence is rising with global temperatures
Peter Schwartzstein
Climate change doesn't just melt ice caps. It also fuels conflict, corruption and division worldwide, explains TED F...

Break the bad news bubble (Part 1)
Angus Hervey
We're stuck in a bad news bubble, says Angus Hervey, founder of Fix the News, an independent publication that r...

War journalism should be rooted in empathy – not violence
Bel Trew
We need journalism that moves beyond a constant focus on violence and honestly depicts the full impact of war, in and ...

3 Ideas for communicating across the political divide
Isaac Saul
How does language shape our politics? Journalist Isaac Saul explores how subtle word choices can inhibit productive di...

The good news you might have missed
Angus Hervey
Whether or not you believe the world is doomed might depend on where you get your news, says journalist Angus ...

Related Talks given by Neo4j



01. Realizzazione App

Realizzare questa app non è stato semplice: mettere in pratica quanto spiegato dall'ingegnere Bonacina ha richiesto diversi approfondimenti e studio di gruppo.

Dopo varie prove e ricerche, siamo riusciti a tradurre nella pratica i concetti affrontati a lezione, costruendo un'app Flutter funzionale



UNIVERSITÀ
DEGLI STUDI
DI BERGAMO

Possibili Evoluzioni

Capitalizzando sulle attuali implementazioni e analizzando il progetto nell'insieme, le prossime evoluzioni mirano a ottimizzare, scalare e arricchire ulteriormente il sistema.

Talk in diretta con generazione automatica di appunti AI

Consentire agli utenti di personalizzare la schermata principale scegliendo quali categorie, tag o talk mettere in evidenza, rendendo l'esperienza più su misura per ciascun utente.

Sezione di chat AI per domande sui talk

Integrare una chat AI dove l'utente può fare domande sui talk e ricevere risposte personalizzate.

Aggiungere una schermata di onboarding o tutorial

Introdurre una breve guida iniziale che spieghi le funzionalità principali dell'app agli utenti al primo avvio.



UNIVERSITÀ
DEGLI STUDI
DI BERGAMO

Team TEDxGRAPH



**Davide
Bonsembiante**

Mat. 1086863



**Thomas
Paganelli**

Mat. 1085805



**Lorenzo
Gallizioli**

Mat. 1087404